

Confirmed Decisions (2026-04-11)

1. **Sequencing:** Sovereign inference cutover FIRST, per-publisher agent runtime SECOND. Unlocks the "no vendor LLM" pitch as soon as the migration lands, before any new UI.
2. **Hosting:** Together AI dedicated endpoint in Phase 1; revisit colocated H100 in Phase 3 once usage is predictable and we've hit ~10 papers.
3. **Fallback policy:** HARD BAN. After migration, zero Anthropic/OpenAI in publisher-touching paths. CI check fails build if anthropic.Anthropic or openai.OpenAI is imported in those modules. No emergency fallback — if self-hosted goes down we return a maintenance message, not a vendor answer.

Context

Trevor is asking two linked questions:

1. **Strategic:** What if every publisher in Grand Network gets its *own* agent — potentially thousands of agents across the network? How powerful would that be, and what's the business case?
2. **Constraint:** Publishers are (rightly) terrified of feeding their archives to the same LLM vendors that are destroying their referral traffic. Trevor's stance is explicit: **no large LLM may train on our data, ever**. This is both a technical requirement and a publisher-trust narrative.

Why this matters now. The Grand Network thesis (project_advertising_brain_vision.md) is that "the moat is the data" — structured hyperlocal content that Google/OpenAI/Yelp cannot reach. If that data leaks into a vendor's training set, the moat is gone. Worse: we can't sell publishers on joining the network if our answer to "where does my content go?" is "OpenAI and Anthropic, but don't worry about it."

Current reality (verified 2026-04-11). The publisher-touching inference today runs through:

Component	Vendor	Model	File
Chat / RAG	Anthropic	Claude Sonnet 4	src/query_engine.py:68-87, src/search_agent.py:76,221
Metadata extraction	Anthropic	Claude	src/metadata_extractor.py:75, metadata_extractor_ads.py, metadata_extractor_events.py

Component	Vendor	Model	File
Query transformation	Anthropic	Claude	src/query_transformer.py:72
Vision extraction	OpenAI	GPT-5.4	src/modules/extraction/pipeline_vision.py:78-84
Directory enrichment	DO Gradient	Qwen3-32B	src/modules/directory/enrichment.py:161-212
Embeddings (safe)	local	all-MiniLM-L6-v2	src/query_engine.py:55
Web search (safe)	Brave	N/A	src/modules/directory/enrichment.py:58-113

Anthropic and OpenAI both contractually **do not train on API traffic** under their commercial ToS. That's the reality. But the *perception* — in an editor's newsroom, pitching a skeptical publisher — is what kills deals. And ToS can change. Trevor's "no large LLMs training on my data" stance is correct even if current ToS technically satisfy it.

The Per-Publication Agent Concept

Re-framing: the question isn't "one agent vs thousands." It's **how much is scoped per publisher, and how much is shared infrastructure.**

Today's architecture already filters ChromaDB by publisher metadata (src/query_engine.py + ArticleSearch._query_collection). That's a *soft* multi-tenant chatbot — same system prompt, same tools, same model, different index slice. The per-publication agent vision goes further:

A "publisher agent" = a config record that binds together:

- A **system prompt** with the paper's voice, editorial stance, disclosure rules, blocked topics
- A **scoped retrieval scope** — only this publisher's articles/ads/events/directory (plus federation opt-ins)
- A **tool set** the editor chose — e.g. Pipestone gets historical_search + directory_search + events_search; a paper without events disables that tool
- A **memory namespace** — per-user conversation history isolated per publisher

- An **audit log** the publisher can read anytime: "here's every question readers asked your agent and every answer we gave"
- Optionally, a **fine-tuned small model** trained only on their archive (premium tier)
- A **federation contract**: what other agents in the network may ask this agent about, and what it will share back

Sub-agents inside each publisher. A paper isn't monolithic — news, sports, obits, classifieds, directory, archive-research are different jobs. Each can be its own sub-agent with a specialized prompt and tool set, coordinated by a router agent at the paper level. At 50 papers × ~5 sub-agents, that's ~250 agents. At 150 papers (10-year vision) × 5 = 750. "Thousands" is achievable once you add ad-buyer agents, business-dashboard AI assistants, etc. But the count isn't what makes it powerful — the **scoping and federation** is.

Federation — the "Main Street" layer. A network-level router agent receives a cross-paper query ("what's happening with wind energy in southern MN?") and dispatches tool-calls to each publisher agent. Each publisher agent answers from its own scope and can *refuse* to surface certain content (e.g. editorials, opinion pieces). The router synthesizes with clear source attribution. **This is the architecture that turns "moat is the data" into "moat is a living answer layer only we can query."**

Benefits — Publisher Side

1. **Data sovereignty as a deliverable.** "Your content never touches OpenAI or Anthropic. Ever. Here's the audit log." Every other AI pitch a publisher gets this year is the opposite of this.
2. **Brand voice preservation.** Pipestone Star agent sounds like the Pipestone editor; Cottonwood Citizen sounds like theirs. Not a generic ChatGPT wrapper with a logo.
3. **Editorial control without code.** Editor-facing UI to tune the system prompt, block topics, require source citations, set tone. No engineering tickets to change behavior.
4. **New reader touchpoint, zero site rebuild.** Agent embeds in the paper's existing site (Door 1 of the Two-Door model). They don't have to migrate anything.
5. **Reader intelligence they can't get otherwise.** "Here are the top 20 questions your readers asked this week that your coverage didn't answer" — a literal content-planning tool that falls out of the agent for free.

6. **Revenue share across all 7 lines of the stack** (project_advertising_brain_vision.md), not just ad revenue. The agent is the surface that makes Network Pass, LLM licensing, Sponsored Answers, and Business Dashboard all work.
7. **"Their own AI" ownership narrative.** Pride of ownership is real. Publishers who believe the agent is *theirs* will defend it, promote it, and renew on it.
8. **Compliance posture.** When the AP/NYT lawsuits shake out, publishers with a documented "our content is not in any LLM training set" posture are legally insulated.

Benefits — Grand Network Side

1. **The moat compounds.** Not just raw articles anymore — trained agents, configured prompts, per-publisher memory, audit trails. Switching cost goes from "export ChromaDB" to "give up the whole agent that's been learning our voice for 18 months."
 2. **Cleaner billing unit.** Each agent is a metered object — queries, tool calls, tokens. Network Pass, Business Dashboard, and Sponsored Answers all meter on agent usage, which drops straight into the 60/40 or 70/30 splits from the revenue stack.
 3. **Defensibility at the publisher door.** When an OpenAI rep walks into Windom next year offering "ChatGPT for your archive" — the answer is "we already have our own agent, trained only on our content, and it doesn't feed any big AI company." Game over for the salesperson.
 4. **LLM licensing becomes more valuable, not less.** We're not selling OpenAI "raw articles" (commodity). We're selling access to the federated *answer layer* — the queryable brain — with per-publisher revenue share and usage caps. That's a novel product category.
 5. **Cross-agent federation IS the network effect.** Every new publisher makes the Main Street router measurably smarter. Quantifiable to prospective publishers: "at 10 papers the router answered 67% of regional queries; at 30 papers, 89%. Your paper makes the next paper easier to sell."
 6. **Per-publisher analytics become a product.** We already have the plumbing in src/modules/costs/tracker.py — extending to per-publisher query logs is near-free and becomes a dashboard tab the publisher pays for.
 7. **White-label pathway.** Once the per-publisher agent architecture is clean, the same runtime can be sold to *other* newspaper networks as Grand Network OS. (Referenced in the v1.1+ vectors of project_main_street_os.md.)
-

The "No Large LLM Training" Constraint — What It Actually Means

Trevor's stance is absolute. There are three layers to it:

Layer 1 — Contractual. API calls to Anthropic/OpenAI under commercial ToS do not train on customer data today. Zero-retention endpoints exist. Technically compliant. But:

Layer 2 — Perceptual. A publisher newsroom does not parse ToS. If the answer to "who sees our content?" includes the word "OpenAI," the deal is dead. The sovereign-AI story only works if *no commercial vendor LLM is in the inference path at all* for publisher content.

Layer 3 — Structural. Vendor ToS can change. Lawsuits can force disclosures. Enterprise agreements can be renegotiated unfavorably. The only durable answer is infrastructure we control. "We run the models" is a permanent artifact; "the vendor's current ToS says they won't" is a temporary one.

Therefore: all publisher-touching inference must move to self-hosted or dedicated-tenancy open-weight models. This excludes Anthropic Claude and OpenAI GPT-5.4 from the hot path for chat, metadata, search, query transform, and vision extraction. It can *keep* Brave Search (search, not training), local embeddings (already safe), and anything that isn't sending publisher content to a third party that could train on it.

★ Insight ————— The migration is an opportunity, not just a cost. The "our data never leaves" story is a marketing asset worth multiples of the infrastructure bill — it becomes the single most-quoted line in the publisher pitch and the strongest rebuttal to every OpenAI competitor.

Best-Model Recommendations (Sovereign / Open-Weight Only)

Chat / RAG / Agent reasoning

Tier	Model	Strength	Serving
Flagship (federation router, complex reasoning)	Llama 3.3 70B Instruct OR Qwen 2.5 72B Instruct	Both beat Claude 3.5 Sonnet on most public benchmarks; strong tool-use	Dedicated GPU endpoint
Workhorse (per-publisher chat)	Llama 3.3 8B OR Qwen 2.5 14B	Cheap, fast, good enough for 90% of reader queries	Shared GPU endpoint

Tier	Model	Strength	Serving
Emerging option	DeepSeek V3 / R1	Strong reasoning on cheap hardware	Dedicated endpoint — vet licensing terms carefully

Start with Llama 3.3 70B for everything; step down to 8B for simple queries after we instrument quality metrics. Qwen is a strong alternative but has China-origin licensing optics some publishers may ask about — Llama is the safer default for narrative.

Vision / Document Extraction (replaces GPT-5.4)

This is the hardest migration. GPT-5.4 at \$0.04/page is genuinely excellent (history in memory: Claude hallucinated, Ollama was undersized, GPT-5.4 won). Open alternatives:

Model	Notes
Qwen 2.5-VL 72B	Currently the strongest open vision-language model on document understanding. Matches or beats GPT-4o on many doc-parsing benchmarks. The front-runner.
InternVL 2.5 78B	Comparable to Qwen 2.5-VL; slightly weaker OCR.
Llama 3.2 90B Vision	Weaker on dense document extraction than Qwen — not recommended for this use.

Expectation: a ~5-15% accuracy regression vs GPT-5.4 on the hardest pages, closing quickly as open models iterate. Mitigate by (a) tighter structured-output prompting, (b) a post-pass text normalizer, (c) publisher-visible "extraction confidence" flag in the review UI.

Embeddings (upgrade, not replace)

Currently all-MiniLM-L6-v2 (384 dim, local). Upgrade:

- **BGE-M3** — multilingual, 1024 dim, longer context, strong retrieval quality. Local. Free.
- Add a **reranker** (BGE-reranker-v2-m3) as a second stage before LLM synthesis. This is the single highest-ROI quality win available right now and has nothing to do with vendor lock-in.

Metadata / query transformation / directory enrichment

These are structured-output tasks (JSON extraction, query rewriting, business enrichment). The Llama 3.3 8B workhorse handles all of them well with JSON-mode prompting. No need for separate models; consolidate onto one inference endpoint.

Tech Stack Recommendation

Hosting options for open-weight models

Option	Pro	Con	Fit
Dedicated endpoints on Together AI / Fireworks AI / Anyscale	No hardware, per-second billing, no-training contracts available, BAAs, SLAs	Still a vendor in the path (but no-train contractually)	Best near-term
Colocated GPU box (1-2× H100 or 2× A100 80GB)	Total sovereignty; publishers can physically visit the hardware. Strongest narrative. Pays back in ~2 years at network scale.	~\$25-40k capex + ~\$3k/yr power/colo. Ops burden. Single-point-of-failure unless you buy two.	Best 12-month target
Runpod / Lambda / CoreWeave per-hour rental	Cheapest experimentation, spin up/down	Cold-start latency; still vendor-ish	Best for vision pipeline batch jobs
DigitalOcean Gradient (current for Qwen3-32B enrichment)	Already wired up	Shared tenancy, vendor training posture unclear — fails the narrative test for publisher-facing chat	Acceptable as interim for directory enrichment only
Railway self-managed GPU	Simplest integration with current deploy	Expensive per hour, limited GPU options	Not recommended

Recommended sequence: Together AI dedicated endpoint (Phase 1, 4-6 weeks) → evaluate usage and costs → colocated H100 box (Phase 3, 3-6 months) once we've hit ~10 publishers and predictable query volume.

Agent framework

Keep it boring. The codebase already uses LlamaIndex patterns and a simple tool-use loop (src/search_agent.py). **Do not** adopt LangGraph, CrewAI, AutoGen, or any multi-agent orchestration framework — they're overkill, add lock-in, and complicate audit logging. Instead:

- **Per-publisher agent = YAML/JSON config** loaded at query time from a new publisher_agents table (or .yaml file committed per publisher for now).
- **Agent runtime = simple Python loop**: load config → scope retrieval → pick tools from a registry → call LLM with tool-use → execute tools → log everything → return.
- **Federation router = another instance of the same runtime** with a call_publisher_agent(slug, question) tool that invokes the sub-agents.

Total new code for the runtime: under 500 lines. The hard part is the config schema and the audit plumbing, not the orchestration.

Data isolation (hard requirement, already mostly there)

- ChromaDB filter by publisher metadata is already enforced in ArticleSearch._query_collection() — extend with an assertion layer that raises if a query ever returns a chunk tagged with a different publisher than the agent that issued it. Defense in depth.
- Separate conversation history tables per publisher (or tag existing conversations with publisher_id if not already).
- Audit log table: agent_queries(id, publisher_id, user_id, question, retrieved_chunk_ids, tool_calls_json, llm_provider, llm_model, response, latency_ms, cost_cents, timestamp). Publisher can query this via a dashboard — "prove to me you're not feeding my content to OpenAI" becomes a SQL query they can see.

Observability

Add **Langfuse** (open-source, self-hosted, MIT-licensed) or **Phoenix by Arize** (open-source, Apache 2). Both give per-trace LLM observability, cost tracking, and eval dashboards. Self-hosted means no trace data leaves our infrastructure. Publisher can have a read-only login: "show me every trace from my agent." Extraordinary trust signal.

Web search tool (unchanged)

Keep Brave Search for directory enrichment and any agent tool that needs the web. Brave is privacy-first, doesn't retain content for training, and has the best independent crawl.

Cost Analysis

Current inference cost (50 publishers, rough projection):

- Vision (GPT-5.4): $\$0.56/\text{edition} \times 52 \times 50 = \sim\$1,460/\text{yr}$
- Chat (Claude Sonnet via Anthropic): $\sim\$20/\text{mo}/\text{paper peak} = \sim\$12,000/\text{yr}$
- Metadata/query-transform (Claude): bundled into chat $\sim\$2,000/\text{yr}$
- Enrichment (Qwen3-32B on DO Gradient): $\sim\$0.004/\text{business} \times 500 \times 50 = \sim\$100/\text{yr}$
- **Total at 50 papers: $\sim\$15,500/\text{yr}$**

Self-hosted projection (Phase 1: Together AI dedicated endpoint):

- Llama 3.3 70B dedicated: $\sim\$2/\text{hr} \times 24 \text{ hrs} \times 365 = \sim\$17,500/\text{yr}$
- Qwen 2.5-VL 72B dedicated (vision, batch during upload): $\sim\$1.50/\text{hr} \times 6 \text{ hrs/day (burst)} \times 365 = \sim\$3,300/\text{yr}$
- Llama 3.3 8B shared (workhorse): $\sim\$0.30/\text{hr} \times 24 \times 365 = \sim\$2,600/\text{yr}$
- **Total at 50 papers: $\sim\$23,400/\text{yr}$**

Self-hosted projection (Phase 3: colocated H100):

- 1x H100 colocation: $\sim\$25\text{k}$ amortized over 3 years = $\sim\$8,300/\text{yr}$
- Power + bandwidth: $\sim\$3,000/\text{yr}$
- 1 backup GPU rental for failover: $\sim\$4,000/\text{yr}$
- **Total at 50 papers: $\sim\$15,300/\text{yr}$** — *about the same as current vendor costs, but at 100+ papers it drops below vendor cost on a per-publisher basis and scales flat.*

Unit economics win. At current costs, inference is $\sim\$310/\text{yr}/\text{publisher}$. Even at the most expensive phase, it's $\sim\$470/\text{yr}$. Against a Business Dashboard ARPU of $\sim\$100/\text{mo} \times 60\%$ publisher share $\times \sim 30\%$ network take = real inference budget per publisher of many thousands per year. **Inference cost is not the constraint; trust and quality are.**

★ Insight ————— The self-hosted path is roughly cost-neutral even at Phase 1 because GPUs run whether traffic comes or not. This means we should aim for *high query volume per publisher* (embed the agent everywhere) rather than minimize calls — the marginal cost of another query is $\sim$ zero once the endpoint is up.

Things to Think Through (Risks & Open Questions)

1. **Vision quality gap.** GPT-5.4 today is genuinely better than Qwen 2.5-VL on the hardest pages (dense IDML, weird column wraps). Need a measured comparison before committing. *Mitigation:* dual-run during transition, publish side-by-side results.
 2. **Cold-start latency.** Dedicated endpoints can be slow on first query. *Mitigation:* warm pool + query-time loading indicators + async pre-warm on page load.
 3. **Open-model tool-use weakness.** Llama/Qwen tool-use is maturing but still weaker than Claude on edge cases (nested calls, retries). *Mitigation:* JSON-mode enforcement, constrained decoding via outlines or guidance libraries.
 4. **Publisher data-portability guarantees.** If an agent is "theirs," what happens when they leave the network? Need an explicit contract clause and a one-click export (archive chunks + agent config + audit log as a tarball).
 5. **Per-publisher fine-tuning temptation.** Huge cost unless carefully scoped. *Recommendation:* start with prompt-level personalization only; offer fine-tuning as a premium upgrade once 2-3 papers request it and are willing to pay.
 6. **robots.txt and crawler blocking.** The agent's public pages should aggressively block GPTBot, ClaudeBot, PerplexityBot, Google-Extended. Two reasons: publisher trust, and denying free training data to the vendors we're replacing.
 7. **Existing Gradient/DO Qwen3 path.** Directory enrichment currently uses DigitalOcean Gradient. Vet DO's training policy explicitly; if ambiguous, migrate enrichment onto the same Together/self-hosted endpoint.
 8. **Hallucination liability.** Per-publisher agents that cite sources wrongly create legal risk for the *publisher*, not just us. Need a verified-citation layer — the agent must only cite chunks it actually retrieved, enforced at the response-generation stage, not trusted to the model.
 9. **Model-update cadence.** Self-hosted means we pin versions. That's good for reproducibility but requires a rollout process. Use a staging endpoint + side-by-side eval before promoting.
 10. **Capacity planning.** A single dedicated 70B endpoint can handle ~10-30 queries/second. That's enough for 50 papers. At 150 papers we need horizontal scaling — plan for it but don't build it early.
-

Phased Path (high-level, no implementation detail)

Phase 0 — Decision & narrative (this week). Trevor confirms the sovereign-AI stance and the per-publication agent direction. Rewrite the publisher pitch deck to lead with "your content never touches a vendor LLM." This is free marketing leverage we can use *before* the migration is complete.

Phase 1 — Sovereign inference cutover (4-6 weeks). [FIRST — CONFIRMED]

- Stand up Together AI dedicated endpoint for Llama 3.3 70B + Qwen 2.5-VL 72B with an explicit no-training contract.
- Add a LLM_PROVIDER=together branch to src/query_engine.py, src/search_agent.py, metadata/query modules, and src/modules/extraction/pipeline_vision.py.
- Measure quality regression on a held-out set of real queries and real pages. Publish results internally.
- Migrate in order: (1) directory enrichment (lowest risk), (2) metadata extraction, (3) chat/search, (4) vision.
- **Hard ban at end of phase:** remove all anthropic.Anthropic and openai.OpenAI imports from publisher-touching modules. Add CI check. No emergency fallback — if the self-hosted endpoint is down we return a maintenance message, not a vendor answer.

Phase 2 — Per-publication agent runtime (4-6 weeks).

- Design publisher_agents config schema (YAML files committed per publisher, promoted to DB table later).
- Simple agent runtime in ~500 lines: load config → scoped retrieval → tool registry → LLM loop → audit log.
- Per-publisher admin UI page: edit prompt, toggle tools, view audit log.
- Migrate Cottonwood and Pipestone onto the new runtime as proof.
- Add BGE-M3 embeddings + BGE-reranker-v2-m3 as a retrieval quality win.

Phase 3 — Federation router + observability (4-6 weeks).

- "Main Street" router agent that dispatches cross-paper queries to publisher sub-agents via tool calls.
- Self-hosted Langfuse for per-trace observability; publisher read-only login.
- Per-publisher data-portability export (tarball).

- Start evaluation of colocated H100 option once usage is predictable.

Phase 4+ — Optional premium features.

- Per-publisher fine-tuning (LoRA) offering.
- Sub-agent specialization (news/sports/obits/directory).
- Voice-interface agent frontend.
- White-label packaging for other newspaper networks.

Critical Files to Modify (forward-looking reference, nothing touched yet)

- `src/core/config.py` — add together/self_hosted provider options, endpoint URLs, model IDs
- `src/query_engine.py:68-87` — provider branch for together
- `src/search_agent.py:76,221` — same
- `src/metadata_extractor.py`, `metadata_extractor_ads.py`, `metadata_extractor_events.py` — same
- `src/query_transformer.py:72` — same
- `src/modules/extraction/pipeline_vision.py:78-84` — add `VISION_PROVIDER=qwen_vl` branch calling Together endpoint
- `src/modules/directory/enrichment.py:161-212` — vet DO training stance, migrate if needed
- **NEW:** `src/modules/agents/runtime.py` — per-publisher agent loop
- **NEW:** `src/modules/agents/config.py` — config schema, loader, validator
- **NEW:** `src/modules/agents/audit.py` — audit log writer + query API
- **NEW:** `src/modules/agents/federation.py` — router for cross-publisher queries
- **NEW:** `configs/agents/{publisher_slug}.yaml` — per-publisher agent configs (one file per paper to start)
- `src/admin_frontend/routes.py` — new routes for per-agent config UI
- **NEW:** `src/admin_frontend/templates/agent_editor.html` — UI to edit system prompt, tools, audit log view

- `src/modules/costs/tracker.py` — add per-agent cost attribution column

Reuse, don't rebuild: existing ChromaDB publisher filtering (`src/query_engine.py`, `ArticleSearch._query_collection`), existing `costs/tracker.py` telemetry pattern, existing `editions.is_current` boost logic, existing `historical_search` tool.

Verification Plan

End-to-end test before promoting any phase:

1. **Sovereignty audit.** Grep the running codebase for any call to `anthropic`, `openai`, `OpenAI`, or equivalent SDKs in publisher-touching paths. Target: zero hits after Phase 1 (except explicit fallback paths behind env var). Automate as a CI check.
2. **Quality regression.** Golden query set: 30 real chatbot queries against Cottonwood + Pipestone archives, with expected answers. Run through old (Claude) and new (Llama 3.3 70B) and diff. Accept if >90% of answers are equivalent quality; flag the rest for targeted prompt tuning.
3. **Vision quality regression.** Same idea for extraction — 10 real edition pages with human-verified extractions, compare Qwen 2.5-VL output against GPT-5.4. Accept ~90% match or better; investigate gaps.
4. **Per-publisher isolation test.** Fire a query to Cottonwood's agent with a distinctive Pipestone-only keyword. Assert no Pipestone chunks returned in retrieval. Assert no Pipestone chunks in audit log. Repeat for every publisher pair.
5. **Live site smoke test.** Ask "what happened in Windom this week?" on `/windom` — expect current-edition stories from Cottonwood, nothing else. Ask same on `/pipestone` — expect Pipestone stories only. Verify audit log entries.
6. **Cost ceiling alert.** Set a daily inference-cost threshold in `src/modules/costs/tracker.py`; page if exceeded.

Everything here is blocked on Trevor's explicit go-ahead before any code runs.